

人工智能 Python 编程

机器学习

Chengwei QIN, AI Thrust

面向 AI 工作流的库

- NumPy、SciPy、Pandas、Matplotlib 与 Seaborn

机器学习基础

- 核心术语、主要类别、核心流程、主要挑战



面向 AI 工作流的库

- NumPy
- SciPy
- Pandas
- Matplotlib 与 Seaborn

把 Python 接到 AI 相关计算上

- NumPy: 快速 N 维数组、向量化、线性代数（向量与矩阵运算）
- SciPy: 建立在 NumPy 之上的算法: optimize（寻找最优模型参数）、stats（统计检验）、sparse（稀疏矩阵）、signal（信号处理）
- Pandas: 表格数据的读写、清洗、变换、分组与连接
- Matplotlib 与 Seaborn: 出版级绘图, 用来呈现结果

为什么重要?

- 性能（向量化的 C/Fortran 内核）、易用性（清晰 API）、生态（可与 scikit-learn / PyTorch 协作）
- 这些是后续所有 AI/ML 工作都会反复用到的基础。

Library	Core Idea	Primary Data Type	Typical Tasks
NumPy	Vectorized numerical computing	ndarray	Element-wise ops, broadcasting, LA, random
SciPy	Algorithms on arrays	ndarray/sparse	Optimize, stats tests, sparse ops, FFT
pandas	Tabular data wrangling	DataFrame/Series	I/O, cleaning, merge/join, groupby, features
Matplotlib	Visualization backend	n/a	Line/scatter/bar, images, layout, export

Table: High-level comparison for when to reach for each library.



NumPy

为什么用 NumPy?

问题：标准 Python 列表很灵活，但数值运算很慢。

- Python 循环有解释器开销，不适合大规模数学计算。
- 列表可混存不同类型，难以做底层优化。

解决方案：NumPy (Numerical Python)

- 提供强大的 N 维数组对象 ndarray。
- 速度来自核心原则：向量化 (Vectorization) 。
- 运算在已编译的 C/Fortran 代码中完成。

NumPy 代码 → 编译后的 C/Fortran 代码 → CPU

- 是 Pandas、scikit-learn、PyTorch 等库的基础。

```
1 # Let's sum the squares of 10 million numbers
2 # Python List
3 %timeit sum([x*x for x in range(10000000)]) # >> ~800 ms
4 # NumPy Array
5 import numpy as np
6 arr = np.arange(10000000)
7 %timeit np.sum(arr * arr) # >> ~15 ms (>50x faster!)
```

核心原则：向量化

向量化是把一次运算作用到整个数组上，而不是用 Python 循环逐个元素处理。

没有向量化（缓慢的 Python 循环）

```
1 c = []
2 for i in range(len(a)):
3     c.append(a[i] + b[i])
```

使用向量化（快速的 NumPy 运算）

```
1 c = a + b
```

为什么更快？

- 循环从 Python（解释执行）移到了 C（预先编译）。
- 对整块数据只做一次操作，减少开销。
- 还能利用现代 CPU 能力（如 SIMD）并行处理多个元素。

核心对象: `numpy.ndarray`

N 维数组是 NumPy 的核心：同一类型数值构成的网格，因此才能向量化。

1D Array (Vector)

1	5	2	7
---	---	---	---

2D Array (Matrix)

$$\begin{bmatrix} 1.0 & 2.0 & 3.0 \\ 4.0 & 5.0 & 6.0 \end{bmatrix}$$

关键属性：

`arr.ndim`：维度（轴）个数

`arr.shape`：每个维度上的长度元组

`arr.size`：元素总数

`arr.dtype`：元素数据类型（如 `int64`、`float64`）

Category	Common Functions & Methods
Creation	<code>np.array</code> , <code>np.arange</code> , <code>np.zeros</code> , <code>np.ones</code> , <code>np.linspace</code> , <code>np.random.rand</code>
I/O	<code>np.save</code> , <code>np.load</code> , <code>np.savez_compressed</code>
Manipulation	<code>.reshape</code> , <code>.T</code> , <code>.flatten</code> , <code>np.concatenate</code> , <code>np.vstack</code> , <code>np.hstack</code> , <code>np.split</code>
Math (ufuncs)	<code>np.add</code> , <code>np.subtract</code> , <code>np.multiply</code> , <code>np.divide</code> , <code>np.sqrt</code> , <code>np.exp</code> , <code>np.sin</code>
Aggregation	<code>.sum</code> , <code>.mean</code> , <code>.std</code> , <code>.min</code> , <code>.max</code> , <code>.argmin</code> , <code>.argmax</code>
Linear Algebra	<code>np.dot</code> , <code>@</code> , <code>np.einsum</code> , <code>np.linalg.inv</code> , <code>np.linalg.eig</code>
Searching/Logic	<code>np.where</code> , <code>np.argsort</code> , <code>np.unique</code> , <code>np.isin</code>
Indexing	Slicing <code>[:]</code> , boolean masking <code>[arr > 0]</code> , fancy indexing <code>[indices]</code>

Table: This is not exhaustive but covers the vast majority of common use cases.

1. 从 Python 列表创建

```
1 # Create a 1D array
2 a = np.array([1, 2, 3, 4])
3 # Create a 2D array and specify data type
4 m = np.array([[1, 2], [3, 4]], dtype=np.float64)
```

2. 使用内置函数（通常更高效）

```
1 # Create an array of zeros (shape 2x3)
2 zeros = np.zeros((2, 3))
3 # Create an array with a range of elements
4 seq = np.arange(0, 10, 2) # [0 2 4 6 8]
5 # Create an array of random values (uniform distribution)
6 rand_arr = np.random.rand(2, 2)
```

其他常用创建函数： `np.full()`、`np.linspace()`、`np.eye()`

保存与加载数组

认真做 AI 时，需要保存模型权重、预处理数据或实验结果。

二进制格式 (.npy)

- 保存单个 NumPy 数组最高效的方式

```
1 data = np.random.randn(100, 100)
2 np.save('my_data.npy', data)
3 loaded_data = np.load('my_data.npy')
```

压缩归档 (.npz)

- 适合把多个数组合到一个压缩文件里。

```
1 weights = np.array([0.5, 0.1, -0.2])
2 biases = np.array([1.2, 0.7])
3 np.savez_compressed('model.npz', w=weights, b=biases)
4 model = np.load('model.npz')
5 print(model['w']) # Access by keyword
```

操作数组形状

重塑形状

- `arr.reshape(new_shape)` 返回新形状的视图。
- 元素总数必须保持不变

```
1 a = np.arange(12)          # Shape (12,)
2 b = a.reshape(3, 4)        # Shape (3, 4)
3 c = a.reshape(2, -1)       # Shape (2, 6), NumPy infers the
                             6
```

增加新维度

- 用 `np.newaxis` 给数组增加一个维度。

```
1 v = np.array([1, 2, 3])    # Shape (3,)
2 col_v = v[:, np.newaxis]   # Shape (3, 1)
```

转置

- `arr.T` 交换数组的轴（例如行变成列）。

拼接与拆分数组

拼接与堆叠

- `np.concatenate([a, b], axis=...)` 沿某个轴连接数组。
- `np.vstack()` 与 `np.hstack()` 分别用于垂直、水平堆叠。

```
1 a = np.array([[1, 2], [3, 4]])
2 b = np.array([[5, 6]])
3 # Concatenate along rows (axis 0)
4 print(np.concatenate([a, b], axis=0))
5 # [[1 2]
6 #   [3 4]
7 #   [5 6]]
```

拆分

- `np.split(arr, num_sections, axis=...)` 把数组切开。

```
1 arr = np.arange(4) # [0 1 2 3]
2 part1, part2 = np.split(arr, 2) # >> [array([0, 1]), array([2, 3])]
```

向量化实战：逐元素运算

这是向量化最直接的例子。
算术运算作用到每个元素上，无需显式循环。

```
1 a = np.array([10, 20, 30, 40])
2 b = np.array([1, 2, 3, 4])
3
4 # Vectorized arithmetic
5 print(a + b) # >> [11 22 33 44]
6 print(a * b) # >> [ 10  40  90 160]
7 print(a / 2) # >> [ 5. 10. 15. 20.]
8
9 # Universal Functions (ufuncs) are also vectorized
10 print(np.sqrt(b)) # >> [1.  1.414  1.732  2. ]
```

这种向量化写法比 **Python for** 循环更干净、也更快。

线性代数：两种乘法

NumPy 中两种基本的数组相乘方式：

1. 逐元素乘积 (*)

- 向量化运算，把相同位置的元素相乘。形状必须相同或可广播。

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} * \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} a \times e & b \times f \\ c \times g & d \times h \end{bmatrix}$$

2. 矩阵点积 (@)

- 标准矩阵乘法。
- 内侧维度必须匹配： $(m, n) @ (n, p) \rightarrow (m, p)$ 。

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} @ \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} (ae + bg) & (af + bh) \\ (ce + dg) & (cf + dh) \end{bmatrix}$$

```
1 A = np.array([[1, 2], [3, 4]]); B = np.array  
  ([1, 1], [1, 1])  
2 print(A * B)    # Element-wise -> [[1 2], [3 4]]  
3 print(A @ B)    # Dot Product  -> [[3 3], [7 7]]
```

广播：让向量化成为可能

广播描述 NumPy 如何对形状不同的数组做向量化运算。

规则：若兼容，NumPy 会“拉伸”较小数组去匹配较大数组的形状，但并不真正复制数据。

```
1 matrix = np.array([[1, 2, 3],  
2                   [4, 5, 6]]) # Shape: (2, 3)  
3  
4 vector = np.array([10, 20, 30]) # Shape: (3,)  
5  
6 # NumPy "broadcasts" the vector to each row of the  
   matrix  
7 result = matrix + vector  
8 print(result)  
9 # [[11 22 33]  
10 #  [14 25 36]]
```

这对常见 AI 任务很关键，例如把偏置向量加到输入矩阵上。

向量化聚合：概括数据

NumPy 提供快速的向量化函数来计算数组摘要，同样避免缓慢的 Python 循环。

```
1 data = np.array([[1, 2, 3], [4, 5, 6]])
2 # [[1 2 3]
3 #  [4 5 6]]
4
5 # Aggregate over the entire array
6 print(data.sum())      # >> 21
7 print(data.mean())     # >> 3.5
8 print(data.max())      # >> 6
9
10 # Aggregate along an axis
11 # axis=0 -> collapse rows (compute along columns)
12 print(data.sum(axis=0)) # >> [5 7 9]
13
14 # axis=1 -> collapse columns (compute along rows)
15 print(data.sum(axis=1)) # >> [ 6 15]
```

进阶：爱因斯坦求和（einsum）

用指标记号定义张量运算的强大小语言：

语法：'sub1,sub2,...->out'，每个操作数都有指标标签。

例子：

```
1 # 'ij,jk->ik' means:
2 # Sum over the common index 'j'
3 A = np.array([[1, 2], [3, 4]])
4 B = np.array([[1, 1], [1, 1]])
5 C = np.einsum('ij,jk->ik', A, B)
```

Main idea: indices that disappear from the output are summed over.

Compute $C = AB$, where $C_{ik} = \sum_j A_{ij}B_{jk}$.
C is $\begin{bmatrix} 3 & 3 \\ 7 & 7 \end{bmatrix}$

其他双矩阵例子：

einsum	notation	Meaning	einsum	notation	Meaning
	'i,i->'	dot product		'ii->'	matrix trace
	'ij,jk->ik'	matrix multiplication		'ij,ij->ij'	elementwise multiplication
	'ij->ji'	transpose		'ij->i'	sum each row

单矩阵例子

Transpose a matrix: `np.einsum('ij->ji', A)`

Sum along rows: `np.einsum('ij->i', A)`

Get the diagonal: `np.einsum('ii->i', A)`

访问数据：索引与切片

类似 Python 列表，但对多维数组更强大。

基本索引与切片

```
1 arr = np.arange(10) # [0 1 2 3 4 5 6 7 8 9]
2 print(arr[5])       # >> 5
3 print(arr[2:5])     # >> [2 3 4]
4 mat = np.array([[1,2,3], [4,5,6], [7,8,9]])
5 print(mat[1, 2])    # Element at row 1, col 2 >> 6
6 print(mat[:2, 1:])  # First 2 rows, from col 1 onwards
```

访问数据：索引与切片

类似 Python 列表，但对多维数组更强大。

布尔索引（向量化选择）

- 一种强大的向量化选择方式。
- 创建布尔掩码，选出满足条件的元素，全程不用循环

```
1 data = np.array([10, -5, 0, 20, -15])
2 print(data > 0)      # >> [True False False True False]
3 print(data[data > 0]) # >> [10 20]
```

视图与副本：关键概念

为了省内存，NumPy 尽可能不复制。

切片是视图

- 切片是原数组数据的视图。
- 修改视图会改到原数组！

```
1 arr = np.arange(5)      # [0 1 2 3 4]
2 view_slice = arr[1:4]   # [1 2 3]
3 view_slice[0] = 99      # Modify the slice
4 print(arr)              # [0 99 2 3 4] <- Original is changed!
```

视图与副本：关键概念

为了省内存，NumPy 尽可能不复制。

显式复制

- 用 `.copy()` 创建新的独立数组。
- 大多数向量化运算（算术、布尔索引）也会产生副本。

```
1 arr = np.arange(5)
2 copy_slice = arr[1:4].copy()
3 copy_slice[0] = 99
4 print(arr) # >> [0 1 2 3 4] <- Original is safe!
```

访问数据：花式索引

用整数数组访问不连续的指定位置。这会得到副本，而不是视图。

一维例子

```
1 arr = np.arange(20, 30) # [20, 21, ..., 29]
2 indices = np.array([0, 3, 8])
3 print(arr[indices]) # >> [20 23 28]
```

二维例子：选出特定行/列

- AI 中常用来从大数据集里取一个 batch

```
1 matrix = np.arange(12).reshape(4, 3)
2 # [[ 0  1  2], [ 3  4  5], [ 6  7  8], [ 9 10 11]]
3
4 # Select rows 0 and 3
5 print(matrix[[0, 3]])
6 # >> [[ 0  1  2]
7        [ 9 10 11]]
8 # Select elements at (1, 1) and (3, 2)
9 print(matrix[[1, 3], [1, 2]]) # >> [4 11]
```

向量化条件逻辑与搜索

用向量化 NumPy 函数做条件逻辑，避免 Python 循环。

向量化 if-else: np.where()

- **np.where(condition, value_if_true, value_if_false)**

```
1 # Replace all negative numbers with 0
2 arr = np.array([10, -5, 0, 20, -15])
3 clipped_arr = np.where(arr < 0, 0, arr)
4 # >> [10  0  0 20  0]
```

找出最大元素的下标: np.argsort()

- 返回把数组排好序所需的下标。
- AI 里常用来找置信度最高的预测。

```
1 # Model predictions (e.g., probabilities for 5 classes)
2 predictions = np.array([0.1, 0.8, 0.05, 0.0, 0.05])
3 # Get the indices of the top 2 predictions
4 top_2_indices = np.argsort(predictions)[-2:] # >> [0 1]
5 print(np.flip(top_2_indices)) # >> [1 0] (highest first)
```

■ 用于特征工程的向量化集合运算

NumPy 提供快速的向量化集合运算，适合数据探索和构造特征。

找出唯一元素

- `np.unique()` 是理解某个特征有哪些类别的基础。

```
1 feature = np.array(['cat', 'dog', 'cat', 'mouse', 'dog'])
2 unique_vals, counts = np.unique(feature, return_counts=True)
3 print(unique_vals) # >> ['cat' 'dog' 'mouse']
4 print(counts)      # >> [2 2 1]
```

成员测试

- `np.isin()` 检查一个数组的元素是否出现在另一个数组中，返回布尔掩码。

```
1 # Create a feature: is the animal a common pet?
2 common_pets = ['cat', 'dog']
3 is_common_pet_mask = np.isin(feature, common_pets)
4 print(is_common_pet_mask) # >> [ True  True  True False  True]
```

■ 面向 AI 的可复现随机性

随机性用于初始化权重、打乱数据和数据增强。可复现对调试至关重要。

现代写法: **np.random.Generator**

- 用种子创建一个生成器实例

```
1 # Create a generator with a seed for reproducibility
2 rng = np.random.default_rng(seed=42)
3
4 # Generate random numbers from a standard normal
   distribution
5 random_weights = rng.normal(size=(2, 3))
6
7 # Shuffle an array in-place
8 data_indices = np.arange(10)
9 rng.shuffle(data_indices) # Always the same with seed=42
```

Category	Common Functions & Methods
Creation	<code>np.array</code> , <code>np.arange</code> , <code>np.zeros</code> , <code>np.ones</code> , <code>np.linspace</code> , <code>np.random.rand</code>
I/O	<code>np.save</code> , <code>np.load</code> , <code>np.savez_compressed</code>
Manipulation	<code>.reshape</code> , <code>.T</code> , <code>.flatten</code> , <code>np.concatenate</code> , <code>np.vstack</code> , <code>np.hstack</code> , <code>np.split</code>
Math (ufuncs)	<code>np.add</code> , <code>np.subtract</code> , <code>np.multiply</code> , <code>np.divide</code> , <code>np.sqrt</code> , <code>np.exp</code> , <code>np.sin</code>
Aggregation	<code>.sum</code> , <code>.mean</code> , <code>.std</code> , <code>.min</code> , <code>.max</code> , <code>.argmin</code> , <code>.argmax</code>
Linear Algebra	<code>np.dot</code> , <code>@</code> , <code>np.einsum</code> , <code>np.linalg.inv</code> , <code>np.linalg.eig</code>
Searching/Logic	<code>np.where</code> , <code>np.argsort</code> , <code>np.unique</code> , <code>np.isin</code>
Indexing	Slicing <code>[:]</code> , boolean masking <code>[arr > 0]</code> , fancy indexing <code>[indices]</code>

Table: This is not exhaustive but covers the vast majority of common use cases.



SciPy

为什么用 SciPy?

问题：NumPy 提供了表示数据的 ndarray，但缺少模型开发与分析所需的更高层算法。

- 如何找到使模型损失函数最小的最优参数？
- 如何做统计检验来比较两个模型的表现？
- 如何高效处理 NLP 中常见的超大但稀疏的数据集？

解决方案：SciPy (Scientific Python)

- 建立在 NumPy 上，补充科学与工程中的关键算法。
- 提供现成工具来实现、优化和评估模型。
- 把复杂数值细节包起来，让你专注 AI 逻辑。

SciPy 按子包组织。对 AI 而言，有几个特别关键。
按需导入具体模块，代码会更干净、更高效。

- `scipy.optimize`
- `scipy.stats`
- `scipy.linalg`
- `scipy.sparse` 稀疏矩阵运算
- `scipy.special` 特殊数学函数
- `scipy.spatial` 空间算法与距离计算
- `scipy.fft` 傅里叶变换
- `scipy.io` 科学数据文件的读写

```
1 from scipy import optimize
2 from scipy import stats
```

■ 优化：最小化损失（`scipy.optimize`）

这可以说是 AI 里最重要的 SciPy 模块。

它寻找使目标函数（即损失函数）最小的参数。

例子：最小化一个简单损失

- 设 $L(w) = (w - 3)^2 + 5$ 。
- 我们要找到使该损失最小的权重 w 。

```
1  from scipy.optimize import minimize
2  import numpy as np
3  # Define a simple convex loss function
4  def loss_function(w):
5      return (w - 3)**2 + 5
6  # Provide an initial guess for the parameter w
7  initial_w = 0.0
8  # Run the minimization algorithm
9  result = minimize(loss_function, initial_w)
10 print(f"Optimal parameter w: {result.x[0]:.4f}") # >> 3.0000
11 print(f"Minimum loss value: {result.fun:.4f}")  # >> 5.0000
```

优化器正确找到 $w = 3$ 使函数最小，这正对应训练一个简单模型。

AI 工具函数（`scipy.special` 与 `scipy.spatial`）

有些模块提供高度专用、高效的函数，是 AI 算法的积木。

`scipy.special`: 数值稳定性

- 提供 `softmax`、`logsumexp` 等常用函数的数值稳定实现，避免大输入（logits）溢出/下溢。

```
1 from scipy.special import softmax
2 logits = np.array([1000, 1001, 1002])
3 # np.exp(logits) would cause an overflow warning
4 stable_probs = softmax(logits)
5 print(stable_probs) # >> [0.09 0.24 0.66]
```

`scipy.spatial`: 距离度量

- 对 k-NN、聚类、相似度搜索等算法很关键。
- `distance.cdist` 能高效计算两个矩阵中所有点对之间的距离。

进阶线性代数 (scipy.linalg)

scipy.linalg 提供高性能线性代数例程，对许多经典 ML 算法必不可少。

例子：求解回归权重

- 这个简单回归例子中，我们求解 $Ax = b$ 得到权重 x 。

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad b = \begin{bmatrix} 5 \\ 11 \end{bmatrix}$$

```
1 from scipy import linalg
2 A = np.array([[1, 2], [3, 4]])
3 b = np.array([5, 11])
4 x = linalg.solve(A, b)
5 print(f"Solution x (weights): {x}") # >> Solution x: [1. 2.]
6 # Verification: A @ x should equal b
7 print(f"A @ x = {A @ x}") # >> A @ x = [ 5. 11.]
```

用 `linalg.solve` 比手动求矩阵逆更数值稳定，也更快。

统计：比较模型（`scipy.stats`）

用于统计分析的综合模块。

AI 中的一个关键用途：判断两个模型的性能差异是否统计显著。

例子：模型准确率的双样本 t 检验

- 我们有两个模型的交叉验证准确率。
- 模型 B 真的更好，还是差异只是偶然？p 值是关键。

```
1 from scipy import stats
2 # 10 CV scores for Model A (e.g., accuracy)
3 model_a_scores = np.random.normal(loc=0.85, scale=0.05, size=10)
4 # 10 CV scores for Model B (with a slightly higher mean)
5 model_b_scores = np.random.normal(loc=0.88, scale=0.05, size=10)
6 t_statistic, p_value = stats.ttest_ind(model_a_scores, model_b_scores)
7 print(f"P-value: {p_value:.4f}") # e.g., P-value: 0.1588
8 if p_value < 0.05:
9     print("The performance difference is statistically significant.")
10 else:
11     print("No significant performance difference was found.")
```

较小的 p 值 (< 0.05) 说明提升很可能是真实的

处理稀疏数据（`scipy.sparse`）

在 NLP（词袋/TF-IDF）或推荐系统（用户-物品交互）等 AI 领域，矩阵极大但几乎全是 0。

存成稠密 NumPy 数组不可行。

例子：压缩稀疏行（CSR）格式

- 只存储非零元素及其下标，节省大量内存。

```
1 from scipy import sparse
2 # A mostly-zero matrix (e.g. a tiny doc-term matrix)
3 M_dense = np.array([[1, 0, 0, 0, 5], [0, 0, 2, 0, 0], [0, 8, 0, 3,
      0]])
4 # Convert to a memory-efficient sparse representation
5 M_sparse = sparse.csr_matrix(M_dense)
6 print(M_sparse)
7 # (0, 0)      1
8 # (0, 4)      5
9 # (1, 2)      2
10 # (2, 1)      8
11 # (2, 3)      3
12 print(f"Memory (dense): {M_dense.nbytes} bytes")
13 print(f"Memory (sparse): {M_sparse.data.nbytes + M_sparse.indices.
      nbytes + M_sparse.indptr.nbytes} bytes")
```

Module	Common AI Use Cases
<code>scipy.optimize</code>	Minimizing model loss functions (<code>minimize</code>), fitting curves
<code>scipy.stats</code>	Hypothesis tests for model comparison (<code>ttest_ind</code>), probability distributions
<code>scipy.linalg</code>	Solving for model weights (<code>solve</code>), matrix decompositions (<code>svd</code>) for PCA
<code>scipy.sparse</code>	Representing text data (TF-IDF) or user-item interaction matrices
<code>scipy.special</code>	Numerically stable activation functions (<code>softmax</code> , <code>logsumexp</code>)
<code>scipy.spatial</code>	Fast distance calculations for k-NN and clustering (<code>distance.cdist</code>)

Table: SciPy provides the high-level algorithms that complement NumPy's array tools.

运用你的 SciPy 知识。

优化：

- 求 $f(x) = (x - 5)^2 + \cos(x)$ 的最小值。
- 用 `scipy.optimize.minimize`，初值 $x_0 = 0$ 。
- 它找到的是全局最小值吗？

数值稳定性：

- 模型输出 logits $z = \text{np.array}([500, 505, 502])$ 。
- 用 `scipy.special.softmax` 计算概率分布。
- 为什么用 `np.exp(z)` 的朴素实现会出问题？

线性代数：

- 用 `scipy.linalg.solve` 求解下列方程组中的 w_1 、 w_2 ：

$$\begin{cases} 3w_1 + 2w_2 = 12 \\ w_1 + 4w_2 = 14 \end{cases}$$



Pandas

为什么用 Pandas?

问题：原始数据很乱。AI 模型需要干净、结构良好的数值数据。NumPy 高效，但不是为加载、清洗和操控表格数据设计的。

- 如何加载并探索混合类型（文本、数值、类别）的数据集？
- 如何处理真实数据中常见的缺失值？
- 如何构造新特征来提升模型表现？

解决方案：Pandas（Python 数据分析库）

- Python 中 AI/ML 数据预处理流水线的基石。
- 提供 DataFrame：直观、强大的表格数据结构。
- 让特征工程、数据清洗和探索性数据分析（EDA）变得直接。

核心：Pandas Series（一维）

Series 是一维数组式对象，可存放任意类型，并带有称为 index 的标签数组。可以把它想成电子表格里的一列。

```
1 import pandas as pd
2 import numpy as np # Often used with pandas
3 s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
4
5 print(s)
6 # a      10
7 # b      20
8 # c      30
9 # dtype: int64
```

既可用标签访问（如 `s['b']`），也可用整数位置（`s.iloc[1]`）。这些标签在计算中同样起作用。

■ 主力：Pandas DataFrame（二维）

DataFrame 是二维、大小可变的表格结构，行和列都有标签。

- 就像电子表格或 SQL 表。
- 也可以看成由多个 Series 组成的字典。

```
1 data = {'State': ['CA', 'TX', 'FL', 'NY'],
2         'Population': [39.2, 29.5, 21.7, 19.3]}
3 df = pd.DataFrame(data)
4
5 print(df)
6 #      State  Population
7 # 0      CA          39.2
8 # 1      TX          29.5
9 # 2      FL          21.7
10 # 3      NY          19.3
```

在 Pandas 里，大多数数据分析任务都会用到 DataFrame。

Pandas 的一大优势是能轻松读写多种格式。

关键函数： `pd.read_csv()`

- 最常见的加载方式。功能强，有很多选项处理不同 CSV 结构。

```
1 # Assume we have a file 'data.csv'
2 # Name, Age, City
3 # Alice, 25, NYC
4 # Bob, 30, LA
5 df = pd.read_csv('data.csv')
6 print(df)
7 #      Name  Age City
8 # 0  Alice   25  NYC
9 # 1   Bob   30   LA
10 # To save a DataFrame back to CSV:
11 df.to_csv('output.csv', index=False)
```

Pandas 还有 Excel（`read_excel`）、SQL（`read_sql`）、JSON（`read_json`）等读取函数。

访问数据：索引与选择

Pandas 提供强大的数据子集选择方式。

关键方法：.loc[] 与 .iloc[]

- df.loc[]: 按标签选择（行标签/列名）
- df.iloc[]: 按整数位置选择
- 布尔索引：按条件过滤数据

```
1 data = {'State': ['CA', 'TX', 'FL', 'NY'],
2         'Population': [39.2, 29.5, 21.7, 19.3]}
3 df = pd.DataFrame(data)
4 # Using the population DataFrame from before
5 # Select a column
6 print(df['Population']) # Returns a Series
7 # Boolean indexing
8 print(df[df['Population'] > 25])
9 #   State  Population
10 # 0    CA         39.2
11 # 1    TX         29.5
```

应用自定义函数: .apply()

更复杂的自定义变换, .apply() 非常有用。
它沿 DataFrame 的某个轴应用函数。
这是特征工程的关键工具。

```
1 # Create a DataFrame
2 data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Score': [85, 95, 60]}
3 df_scores = pd.DataFrame(data)
4 # Define a function to apply
5 def get_grade(score):
6     if score >= 90:
7         return 'A'
8     elif score >= 80:
9         return 'B'
10    else:
11        return 'C'
12 # Create a new 'Grade' column by applying the function
13 df_scores['Grade'] = df_scores['Score'].apply(get_grade)
14 print(df_scores)
15 #      Name  Score Grade
16 # 0  Alice    85     B
17 # 1   Bob    95     A
18 # 2 Charlie    60     C
```

清洗数据：处理缺失值

真实数据常常不完整。

Pandas 用 NaN 表示缺失，并提供处理工具。

```
1 data = {'A': [1, 2, np.nan], 'B': [5, np.nan, np.nan]}
2 df_missing = pd.DataFrame(data)
3 print(df_missing)
4 #      A      B
5 # 0  1.0  5.0
6 # 1  2.0  NaN
7 # 2  NaN  NaN
8 # Fill missing values with the mean of the column
9 df_filled = df_missing.fillna(df_missing.mean())
10 print(df_filled)
11 #      A      B
12 # 0  1.0  5.0
13 # 1  2.0  5.0
14 # 2  1.5  5.0
```

也可以用 `df.dropna()` 丢掉含缺失的行，或用 `df.drop(columns=['B'])` 丢掉整列。

合并 DataFrame: Concat 与 Merge

数据常常分散在多个文件或表里。

Pandas 提供高性能工具来合并它们。

拼接 (pd.concat) : 把 DataFrame 纵向或横向堆在一起。

```
1 df1 = pd.DataFrame({'A': ['A0', 'A1'], 'B': ['B0', 'B1']})
2 df2 = pd.DataFrame({'A': ['A2', 'A3'], 'B': ['B2', 'B3']})
3 result = pd.concat([df1, df2]) # Stacks df2 below df1
4 print(result)
5 #      A    B
6 # 0  A0  B0
7 # 1  A1  B1
8 # 0  A2  B2
9 # 1  A3  B3
```

合并 DataFrame: Concat 与 Merge

数据常常分散在多个文件或表里。

Pandas 提供高性能工具来合并它们。

连接 (pd.merge)： 按公共键做 SQL 风格的 join。

```
1 left = pd.DataFrame({'key': ['K0', 'K1'], 'val1': [0, 1]})
2 right = pd.DataFrame({'key': ['K0', 'K1'], 'val2': [2, 3]})
3 merged = pd.merge(left, right, on='key')
4 print(merged)
5 #    key  val1  val2
6 # 0  K0     0     2
7 # 1  K1     1     3
```

利器：GroupBy 操作

groupby 机制把数据拆成组，对每组应用函数，再把结果合回去。

```
1 data = {'Team': ['A', 'B', 'A', 'B', 'A'],
2         'Points': [10, 12, 12, 15, 13]}
3 df_team = pd.DataFrame(data)
4
5 # Group by team and calculate the sum of points for each
6 team_scores = df_team.groupby('Team').sum()
7 print(team_scores)
8 #           Points
9 # Team
10 # A             35
11 # B             27
```

这对概括数据非常强大，是数据分析的关键部分。

Category	Example Functions & Purpose
I/O	<code>pd.read_csv</code> , <code>df.to_csv</code> : Read from and write to files.
Selection	<code>df['col']</code> , <code>df.loc[]</code> , <code>df.iloc[]</code> : Select columns, rows, or specific data points.
Cleaning	<code>df.dropna()</code> , <code>df.fillna()</code> , <code>df.drop()</code> : Handle missing data or remove data.
Transformation	<code>df.apply()</code> : Apply a custom function to data.
Combining	<code>pd.merge</code> , <code>pd.concat</code> , <code>df.join()</code> : Combine multiple DataFrames.
Grouping	<code>df.groupby('key').sum()</code> : Split-apply-combine operations for aggregation.
Statistics	<code>df.describe()</code> , <code>df.mean()</code> , <code>df.corr()</code> : Compute summary statistics.

Table: Pandas provides a comprehensive toolkit for the entire data analysis workflow.

运用你的 Pandas 知识。

创建与合并：

- 创建一个 grades DataFrame: Name、Subject、Score (至少 5 行)
- 创建一个 students DataFrame: Name、Major
- 把两个 DataFrame 合并成一张表

过滤：

- 选出 Subject 为 Math 的所有行
- 再选出 Score 大于 90 的所有行

变换：

- 对 Score 列用 .apply() 新建 Pass/Fail 列 (例如 score $\geq$ 70 为 Pass)

聚合：

- 用 groupby() 求每个科目的平均分
- 用 groupby() 求每个专业的平均分



Matplotlib 与 Seaborn

为什么用 Matplotlib?

问题：AI/ML 中我们面对高维数据、模型权重和指标。原始数组/表很难直接看懂。

- 探索性数据分析 (EDA)：特征分布、异常值、相关性？
- 训练：模型在学习还是在过拟合？
- 评估：表现如何？哪些类别被搞混？
- 调试：为何出现奇怪预测？模型学到了什么？

解决方案：Matplotlib 与 Seaborn

- Matplotlib：静态、动画、交互式绘图的核心库。
- Seaborn：建立在 Matplotlib 上，让统计图形更简单。
- 可视化贯穿整个 AI 工作流。

第一个 AI 图：跟踪模型训练

深度学习里最常见的图是损失曲线。

我们先从简单的 pyplot 接口开始。

```
1 import matplotlib.pyplot as plt
2 # 1. Sample Training Data (e.g., from Keras/PyTorch)
3 epochs = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
4 loss = [0.8, 0.6, 0.45, 0.35, 0.28, 0.22, 0.18, 0.15, 0.12, 0.10]
5 val_loss = [0.9, 0.75, 0.6, 0.5, 0.45, 0.41, 0.39, 0.38, 0.37, 0.36]
6 # 2. Create Plot
7 plt.plot(epochs, loss, 'bo-', label='Training Loss')
8 plt.plot(epochs, val_loss, 'ro--', label='Validation Loss')
9 # 3. Add Labels, Title, and Legend
10 plt.title("Model Training & Validation Loss")
11 plt.xlabel("Epochs")
12 plt.ylabel("Loss")
13 plt.grid(True)
14 plt.legend()
15 # 4. Show Plot
16 plt.show()
```

更专业的选择：面向对象接口

需要更多控制、尤其是多图时，面向对象（OO）API 更好。这是复杂 AI 可视化的标准做法。

- **Figure**：整张图所在的窗口或页面。
- **Axes**：Figure 里的一张具体图。

```
1 # Data from previous slide...
2 # 2. Create Figure and a single Axes object
3 fig, ax = plt.subplots(figsize=(8, 5))
4 # 3. Plot on the Axes object
5 ax.plot(epochs, loss, 'bo-', label='Training Loss')
6 ax.plot(epochs, val_loss, 'ro--', label='Validation Loss')
7 # Use `set_*` methods for labels and title
8 ax.set_title("Model Loss (OO Style)")
9 ax.set_xlabel("Epochs")
10 ax.set_ylabel("Loss")
11 ax.grid(True)
12 ax.legend()
13 # 4. Show Plot (or save with fig.savefig())
14 plt.show()
```

用直方图看特征分布

理解数据如何分布，对特征工程和模型选择至关重要。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 # Generate skewed data (e.g., feature like 'income')
4 data = np.random.gamma(2, 2, 1000)
5 fig, ax = plt.subplots()
6 # Plot a histogram to see the distribution
7 ax.hist(data, bins=50, density=True, alpha=0.7, label='
    Histogram')
8 ax.set_title("Distribution of a Skewed Feature")
9 ax.set_xlabel("Feature Value")
10 ax.set_ylabel("Density")
11 plt.show()
```

这张图立刻说明该特征不是正态分布，可能需要考虑对数变换。

用 imshow 可视化图像数据

在计算机视觉任务中，imshow 对显示图像、滤波器和特征图必不可少。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 cnn_filter = np.random.randn(7, 7)
4 # For this example, we create a synthetic one
5 digit = np.array([
6     [0, 0, 5, 13, 9, 1, 0, 0], [0, 0, 13, 15, 10, 15, 5, 0],
7     [0, 3, 15, 2, 0, 11, 8, 0], [0, 4, 12, 0, 0, 8, 8, 0],
8     [0, 5, 8, 0, 0, 9, 8, 0], [0, 4, 11, 0, 1, 12, 7, 0],
9     [0, 2, 14, 5, 10, 12, 0, 0], [0, 0, 6, 13, 10, 0, 0, 0]
10 ])
11 # Create a 1x2 subplot figure
12 fig, axes = plt.subplots(1, 2, figsize=(10, 5))
13 # Display the filter
14 im1 = axes[0].imshow(cnn_filter, cmap='viridis')
15 axes[0].set_title('CNN Filter Visualization')
16 fig.colorbar(im1, ax=axes[0])
17 # Display the digit
18 im2 = axes[1].imshow(digit, cmap='gray')
19 axes[1].set_title('MNIST-like Digit')
20 fig.colorbar(im2, ax=axes[1])
21 plt.show()
```

用散点图看关系

散点图帮助我们看两个特征的关系，以及类别是否可分。Seaborn 让这件事很容易。

```
1 import seaborn as sns
2 from sklearn.datasets import load_iris
3 # Load a classic dataset
4 iris = load_iris()
5 X = iris.data
6 y = iris.target
7 fig, ax = plt.subplots(figsize=(8, 6))
8 # Use Seaborn's scatterplot for easy coloring by class
9 sns.scatterplot(
10     x=X[:, 0], y=X[:, 1], # Sepal Length vs Sepal Width
11     hue=iris.target_names[y],
12     palette='deep',
13     ax=ax
14 )
15 ax.set_title("Iris Dataset: Sepal Length vs. Width")
16 ax.set_xlabel("Sepal Length (cm)")
17 ax.set_ylabel("Sepal Width (cm)")
18 plt.show()
```

模型评估：混淆矩阵

混淆矩阵是观察分类器表现的最好方式之一。

Seaborn 的 heatmap 非常适合画它。

```
1 import numpy as np
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 # Example Confusion Matrix (Actual vs. Predicted)
5 # Rows: Actual Class, Columns: Predicted Class
6 cm_data = np.array([[120, 5, 2],
7                     [8, 110, 1],
8                     [1, 3, 95]])
9 labels = ['Cat', 'Dog', 'Bird']
10 fig, ax = plt.subplots(figsize=(7, 5))
11 sns.heatmap(cm_data, annot=True, fmt='d', cmap='Blues',
12             xticklabels=labels, yticklabels=labels, ax=ax)
13 ax.set_title("Confusion Matrix")
14 ax.set_xlabel("Predicted Label")
15 ax.set_ylabel("True Label")
16 plt.show()
```

这张图显示大多数预测正确，但有 8 只狗被预测成了猫。

Category	Function(s)	Common AI Use Case
Setup	<code>fig, ax = plt.subplots()</code>	Standard way to start any complex plot.
Line Plots	<code>ax.plot()</code>	Plotting training/validation loss & accuracy curves.
Scatter Plots	<code>ax.scatter()</code> , <code>sns.scatterplot()</code>	Visualizing feature relationships, model predictions vs. actuals.
Distributions	<code>ax.hist()</code> , <code>sns.histplot()</code> , <code>sns.kdeplot()</code>	Understanding feature distributions for preprocessing.
Matrix/Image	<code>ax.imshow()</code> , <code>sns.heatmap()</code>	Displaying images, feature maps, attention, confusion matrices.
Labels	<code>ax.set_title()</code> , <code>ax.set_xlabel()</code>	Labeling plots for reports and presentations.
Legends	<code>ax.legend()</code>	Differentiating metrics (e.g., train vs. validation).
Output	<code>plt.show()</code> , <code>fig.savefig()</code>	Displaying plots or saving them for publications.

Table: A toolkit for common visualization tasks in Machine Learning.

NumPy: 快速数组运算、向量化、广播、线性代数。

SciPy: 优化、统计、稀疏数据、进阶线性代数。

Pandas: 用 **DataFrame** 清洗、变换、合并、概括数据。

Matplotlib/Seaborn: 用于探索、调试、评估的可视化。

它们共同构成 AI/ML 的 Python 核心生态。



机器学习基础

机器学习系统可按训练时获得的监督类型和数量来分类。

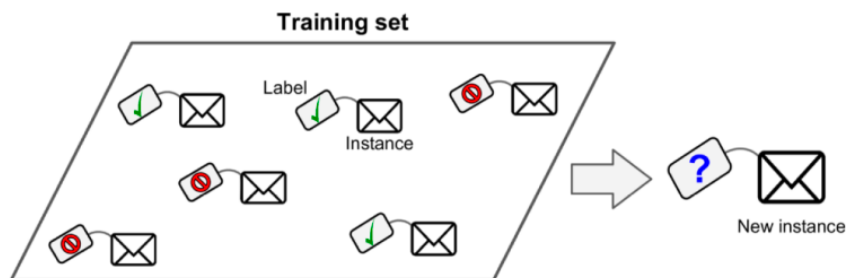
四大类是：

- 监督学习
- 无监督学习
- 半监督学习
- 强化学习

In supervised learning, the training data is a set of n samples $\{(\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})\}$ where each sample includes a feature vector $\mathbf{x}$ and its corresponding label y . The goal is to learn a mapping function f such that $f(\mathbf{X}) \approx \mathbf{y}$.

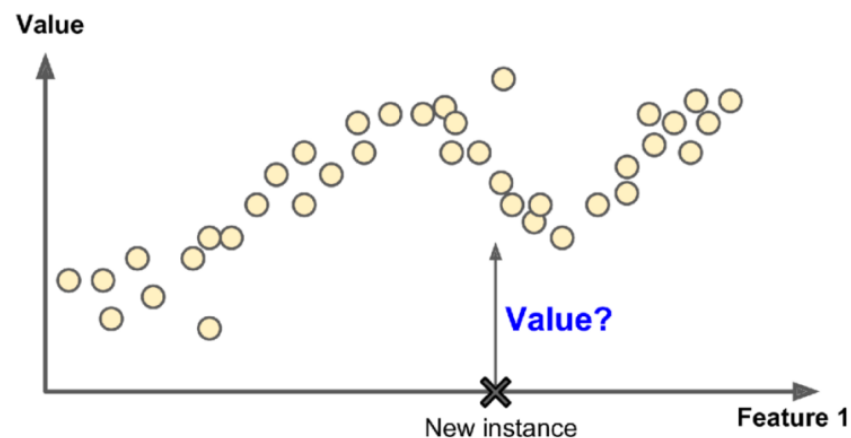
Classification

- The label y is from a finite set of discrete classes.
- Example: Is this email spam ($y = 1$) or not ($y = 0$)?



Regression

- The label y is a continuous numerical value
- Example: What is the price of this car ($y = 25000$)?

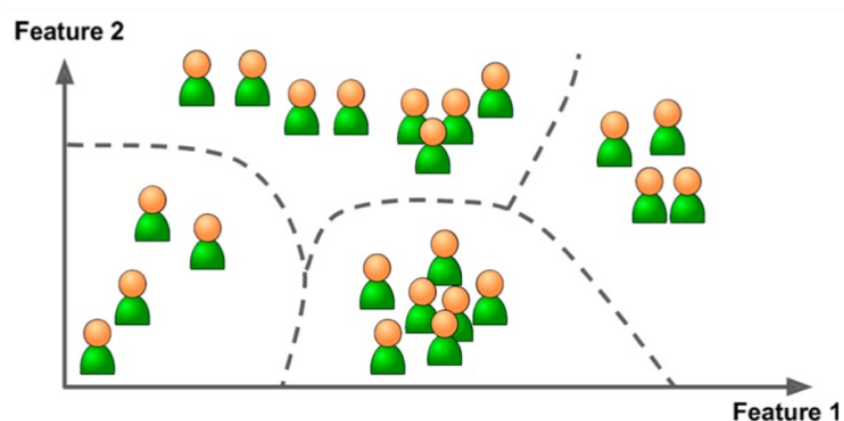


无监督学习

In unsupervised learning, the training data is a set of n samples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}\}$ consisting only of input features **$\mathbf{X}$ without any corresponding labels $\mathbf{y}$** . The system tries to learn the underlying structure or distribution in the data.

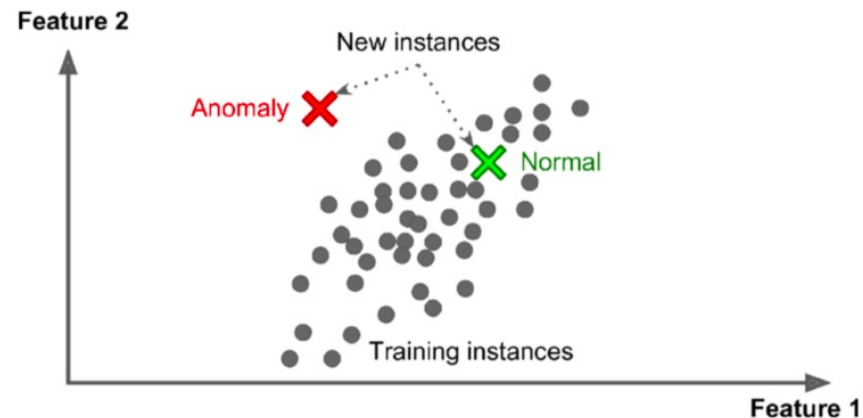
Clustering

- Goal: Group similar instances together based on their features $\mathbf{x}$.
- Example: Customer segmentation based on purchasing behavior.



Anomaly Detection

- Goal: Find instances $\mathbf{x}$ that are significantly different from the rest.
- Example: Fraud detection in credit card transactions.



- 处理部分标注的数据：大量未标注样本，少量已标注样本。
- 有用是因为标注又贵又耗时。
- 例子：有 10,000 张猫狗图片，只有 500 张有标签。半监督算法会同时用这 500 张标注图和 9,500 张未标注图来训练分类器。

- 学习系统称为智能体（agent）：观察环境、选择并执行动作，再获得奖励或惩罚。
- 它必须自己学会最佳策略（policy），以便长期获得最多奖励。
- 例子：训练机器人走路，或训练程序下围棋。
- 注意：强化学习对现代大语言模型（LLM）至关重要，我们后面会详细讲。

数据探索与准备

- 第一步是研究数据集：清洗、分析结构，并为模型做好准备。

模型选择

- 选择适合问题的学习算法，例如回归任务用线性回归，分类任务用分类器。

训练与评估

- 模型从训练数据中学习。
- 再用合适指标评估它学得怎么样。

预测与应用

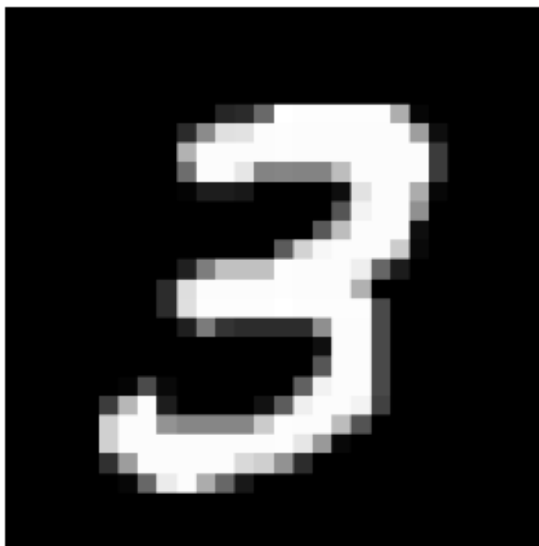
- 最后，把训练好的模型用到新的、未见过的样本上做预测。

例子：手写数字分类

问题

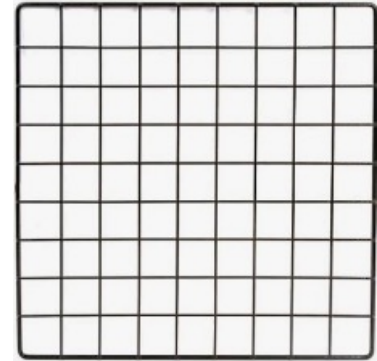
假设有很多手写数字（0-9）图片，每个数字由许多人书写。我们希望写一个机器学习算法，让机器认出任意一张图里的数字。

两张图片的例子



■ Converting an Image to a Numerical Array x

We can divide each MNIST image to grids (28 rows and 28 columns), and use the grayscale (brightness) to represent any square in the grid. Then, each MNIST image is represented with a matrix x of size 28 x 28.



- Each square is called a **pixel** (so each MNIST image has 28x28=784 pixels)
- Each pixel is represented by a number indicating its **brightness**.
- In the original MNIST data, pixel values range from **0 to 255**:
 - 0 → black
 - 255 → white
 - values in between → different brightness levels of gray

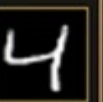
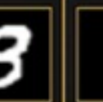
The Training Dataset $\{(\mathbf{x}_n, y_n)\}$

In this example, we use 50,000 training samples from MNIST.

$\{(\mathbf{x}_n, y_n)_{n=1}^{50,000}\}$, where $\mathbf{x}_n$ is the 28x28 matrix representing the pixels of the n^{th} image, and y_n is the value of the digit in the image.

- We convert each y_n to its **one-hot representation** for the later use of classification models.

例子:

Training Image x									
Label y	5	0	4	1	9	2	1	3	1
One-hot Rep.	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

■ The Prediction Model $f_{\mathbf{w}}(\mathbf{x})$

Given any image $\mathbf{x}$, the prediction model $f_{\mathbf{w}}(\mathbf{x})$ is supposed to output the one-hot representation of the digit in the image $\mathbf{x}$. To find such a model, we need to find two things

1. The computational rule in $f_{\mathbf{w}}$;
 2. The value of the parameters $\mathbf{w}$ involved in the computational rule.
-
- **Model Selection** - The first step is done by choosing a model. We choose an FNN classifier for this example.
 - **Model Training (Fitting the Training Data)** – Intuitively (not rigorously), the second step is done by finding the $\mathbf{w}$ value such that

$$f_{\mathbf{w}}(\mathbf{x}_n) \approx \mathbf{y}_n, \quad \text{for each } n \in \{1, 2, \dots, N\}.$$

The Loss Function $L(\mathbf{w})$

The loss function is defined to find $\mathbf{w}$ value such that

$$f_{\mathbf{w}}(\mathbf{x}_n) \approx y_n, \quad \text{for each } n \in \{1, 2, \dots, N\}.$$

For multiclass classification problems, the cross-entropy loss is often used:

$$L(\mathbf{w}) = - \sum_{n=1}^N \left[\sum_{j=1}^K p_{nj} \cdot \ln \hat{p}_{nj}(\mathbf{w}) \right],$$

where $-\sum_{j=1}^K p_{nj} \cdot \ln \hat{p}_{nj}(\mathbf{w})$ reflects the discrepancy between the true value $\mathbf{y}_n$ and the model prediction $f_{\mathbf{w}}(\mathbf{x}_n)$. More specifically,

- $(\mathbf{x}_n, \mathbf{y}_n)$ is the n^{th} sample and $\mathbf{y}_n = [p_{n1}, p_{n2}, \dots, p_{nK}]$ is a one-hot representation of the class label of the n^{th} sample.
- $f_{\mathbf{w}}(\mathbf{x}_n) = [\hat{p}_{n1}, \hat{p}_{n2}, \dots, \hat{p}_{nK}]$ is the model prediction (e.g., $\hat{p}_{n2}$ denotes the model's prediction for $\Pr(n^{th} \text{ sample's label} = 1)$).

The optimizer refers to an algorithm used to find the w value by solving the following optimization problem:

$$\min_w L(\mathbf{w})$$

For neural networks, the popular optimizer used is the stochastic gradient descent (SGD).

After the optimizer finds the value $\mathbf{w}^*$, $f_{\mathbf{w}^*}$ can be tested on the test dataset.

1. Model Selection $f_{\mathbf{w}}$.
2. Setting Loss function $L(\mathbf{w})$.
3. Apply an optimizer to solve the minimization problem $\min_{\mathbf{w}} L(\mathbf{w})$.
4. Test the model $f_{\mathbf{w}^*}$, where $\mathbf{w}^*$ is the parameter value found by the optimizer.

我们来看代码。

- 样本 / 实例：数据集中的一行（例如一套房子、一封邮件）。
- 特征 (x)：用于预测的输入变量（列）。
- 标签 (y)：你想预测的输出变量。
- 模型：学习从特征 x 到标签 y 的映射的算法。
- 训练：模型从数据集中“学习”的过程。
- 推理：用训练好的模型对新样本做预测。

训练数据量不足

- 即使简单问题，通常也需要几千个样本。
- 图像或语音识别等复杂问题，可能需要数百万。

数据不具代表性、质量差

- 训练数据必须能代表你想泛化到的新情况。
- 错误、异常值和噪声会让系统更难发现潜在规律。

无关特征

- “垃圾进，垃圾出。”
- 系统好坏取决于用来训练的特征。
- 特征工程是关键一步。

主要挑战：糟糕的算法

没有一种算法能吃所有问题。

选对算法需要理解各种权衡。

Trade-off 1: Scalability vs. Speed

- For linear regression, the Normal Equation is fast on datasets with a small number of features, but Gradient Descent scales better to large numbers of features.
- Algorithms like Stochastic and Mini-batch GD are much faster than Batch GD on large datasets and support out-of-core learning.

Trade-off 2: Model Assumptions vs. Data Structure

- Linear models assume the data is linear; choosing them for complex, non-linear data will lead to poor performance (underfitting).
- K-Means clustering performs poorly on non-spherical clusters or clusters of varying sizes and densities. A density-based algorithm like DBSCAN would be a better choice for such data.

面向 AI 工作流的库

- NumPy

- SciPy

- Pandas

Library	Core Idea	Primary Data Type	Typical Tasks
NumPy	Vectorized numerical computing	ndarray	Element-wise ops, broadcasting, LA, random
SciPy	Algorithms on arrays	ndarray/sparse	Optimize, stats tests, sparse ops, FFT
pandas	Tabular data wrangling	DataFrame/Series	I/O, cleaning, merge/join, groupby, features
Matplotlib	Visualization backend	n/a	Line/scatter/bar, images, layout, export

- Matplotlib 与 Seaborn

机器学习基础

- 核心术语
- 主要类别：监督 / 无监督 / 半监督 / 强化学习
- 核心流程：数据、模型、训练与评估、预测
- 主要挑战：“糟糕的算法”和“糟糕的数据”

人工智能 Python 编程

感谢聆听!